

1. 项目背景及目标

随着人们生活水平的提高，机动车辆的数量也逐渐增加，2020 年全国的机动车保有总数量为 3.72 亿辆，其中汽车保有量为 2.81 亿辆，占 75.54%。如此庞大的汽车保有量，为交通行业的管理带来了很大的压力。依靠人工进行道路管理，工作量大、出错率高。所以，需要寻找一种有效的方法，提高车辆的管理效率，从而减轻道路行驶的压力。车辆牌照，因其具有唯一性，而成为解决上述问题的关键。

本项目以车辆牌照为依据，基于 matlab 软件设计了车牌识别系统(Vehicle License Plate Recognition, VLPR)，能够检测到受监控路面的车辆并自动提取车辆牌照信息（汉字字符、英文字母、阿拉伯数字）。该系统可用于道路监控、社区车辆登记、停车场车辆登记、高速公路电子收费等场景，为道路交通管理提供便利。

2. 系统说明

本项目所设计的车牌识别系统(VLPR)，主要实现的功能为从图像中提取车牌信息，并输出到操作界面中。该系统输入为含车牌信息的图片，输出为图中车辆的车牌信息。车牌识别系统代码可以分为三部分，分别是图像预处理、图像切割、字符识别（如图 1）。接下来依次对三部分内容进行说明。

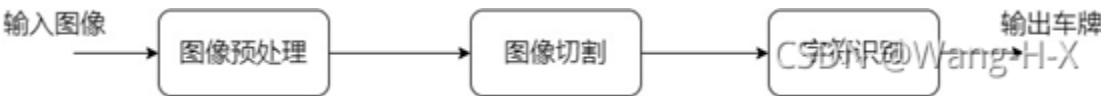


图 1 系统流程图

2.1 图像预处理

由于系统输入的图像信息要素过多，且质量层次不齐，无法直接定位车牌位置并进行切割，所以需要进行预处理。图像预处理的作用是将系统输入图像进行前期处理，为下一步图像切割提供基础。

2.1.1 预处理基本步骤

本系统中图像预处理的主要方法有：图像增强、图像形态学处理、提取特定通道信息等。

1) 图像增强

图像增强主要涉及的操作有提高对比度、图像二值化。

图像对比度是指图像中明暗区域最亮的白与最暗的黑之间不同亮度层级的测量，差异范围越大代表对比度越大，差异范围越小代表对比度越小。在图像预处理的过程

中，提高原始图像的对比度，可以有效地过滤掉无用信息，使目标相关信息视觉上更为明显。提高对比度即提高各阶灰度之间的差异，有利于进行图像二值化处理。

在 matlab 软件中可以通过 `imadjust` 函数实现增强对比度操作。

图像二值化是将图片中像素点的灰度值与所设置的阈值进行比较，若灰度值大于阈值，则将灰度值更新为 255，反之，则将其更新为 0。完成上述操作后的图像，将仅显示黑白两种视觉效果。像素点的灰度仅有 0 与 255 两个数值，故称之为二值化。图像二值化的操作可以通过阈值的设置，有效地过滤掉图像背景、噪声等信息，提取与目标物体有关的有效信息。

在 matlab 软件中可以通过 `im2bw` 函数实现图像二值化操作。

2) 图像形态学处理

图像形态学处理主要涉及的操作有图像膨胀、图像腐蚀、开运算、闭运算、顶帽运算、底帽运算。本系统并未使用后两种运算，故对其不介绍。

图像的膨胀与腐蚀是形态学处理的基本方法。图像的膨胀选取每个像素点邻域内最大的灰度值作为输出灰度值，膨胀过后图像整体亮度会有所提高，图像中较亮的物体尺寸会变大，较暗的物体尺寸会减小甚至消失，从而产生“膨胀”的视觉效果。图像的腐蚀与膨胀相反，选取每个像素点邻域内最小的灰度值作为输出灰度值，腐蚀过后的图像整体亮度会有所减低，图像中较亮的物体尺寸会减小甚至消失，较暗的物体尺寸会增大，从而产生“腐蚀”的视觉效果。

在 matlab 中，可以通过设置参数 `se`，利用 `imdilate` 函数完成图像膨胀的操作，利用 `imerode` 函数完成图像腐蚀的操作。

图像的开运算与闭运算均有腐蚀与膨胀组合而成。开运算是先对图像先腐蚀后膨胀，可以消除亮度较高的细小区域，在纤细点分离物体，对于较大物体可以在不改变其面积的情况下平滑边界。闭运算是先对图像先膨胀后腐蚀，可以填充白色物体内部细小的黑色区域，连接详尽的白色区域。

在 matlab 中，可以通过设置参数 `se`，利用 `imopen` 函数完成图像开运算的操作，利用 `imclose` 函数完成图像闭运算的操作。

3) 提取特定通道信息

计算机对图像的记录有多种方式，最为常见的为 RGB 模型，在本系统中也会运用到 HSV 模型。

RGB 颜色模型即通过红色(Red)、绿色(Green)、蓝色(Blue)三个颜色通道的数据以及它们相互之间的叠加来显示图像。本系统主要用于识别小型汽车，根据《中华人民共和国机动车号牌》(GA 36-2014)中的规定，小型汽车牌照为“蓝底白字”，可根据此信息，利用 RGB 模型，提取图像中的蓝色色块相关要素，从而对图像中的有效元素（车

牌) 进行提取。

HSV 颜色模型是 A. R. Smith 创建的一种颜色模型。将一张图片视为色调(Hue)、饱和度和(Saturation)、明度(Value)三个通道信息的叠加。经过大量实验后发现，S 通道的数据可以有效地显示车牌信息，并将其与周围背景加以区分，所以在进行系统设计时，常进行 S 通道信息的提取。

2.1.2 系统预处理方案

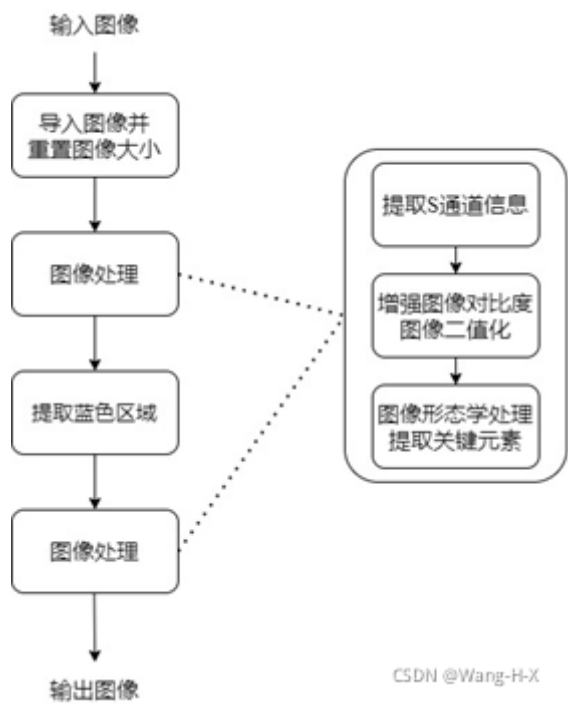


图 2 预处理流程图

本系统图像预处理流程可以分为四步（如图 2 所示）。首先导入图片并重置图片大小，其次进行提取通道信息、图像增强、形态学处理等图像处理操作。完成上述步骤后，根据处理后的图像进行要素提取，提取蓝色区域。再次进行提取通道信息、图像增强、形态学处理等图像处理操作，最终输出图像，完成预处理。

1) 导入图片、重置图片大小

利用 matlab 软件中的 imread 函数进行图片的输入操作。由于导入的图片分辨率不同，不利于后期图像处理及切割。利用 imresize 函数进行图片大小的调整，将所有输入图片调整为 1000*1000 的大小。

```
% 导入图像，将图像像素设置为 1000*1000
image = imread('.\target\4.jpg');
im1 = imresize(image, [1000, 1000]);
pic = im1;
figure(1)
```

```
subplot(3, 3, 1); imshow(im1);
```

2) 第一次图像处理

图像处理主要包含提取 S 通道信息、图像增强、形态学运算三部分。主要运用得函数有：rgb2hsv, imadjust, imclose, imdilate, bwareaopen 等。上述三部分作用已经在 2.1.1 部分进行说明，在此不再赘述。处理结果如图 3 所示。



(a) 输入图像



(b) 提取 S 通道信息



(c) 图像增强



(d) 形态学处理

图 3 图像处理效果图

```
% 提取图像 S 通道信息
```

```
fig1 = rgb2hsv(im1);
```

```
im2 = fig1(:, :, 2);
```

```
subplot(3, 3, 2); imshow(im2);
```

```
% 增强图像对比度，并进行二值化
```

```
im3 = imadjust(im2);
```

```

im4 = im2bw(im3);
subplot(3,3,3); imshow(im4);

% 对图像进行闭运算，并提取二值化图像中的白色部分
se1 = strel('rectangle', [20, 20]);
im5 = imclose(im4, se1);
[m,n] = size(im5);
for i = 1:m
    for j = 1:n
        if im5(i,j) == 0
            pic(i,j,:) = 255;
        end
    end
end
im6 = pic;
subplot(3,3,4); imshow(im6);

```

3) 提取蓝色区域

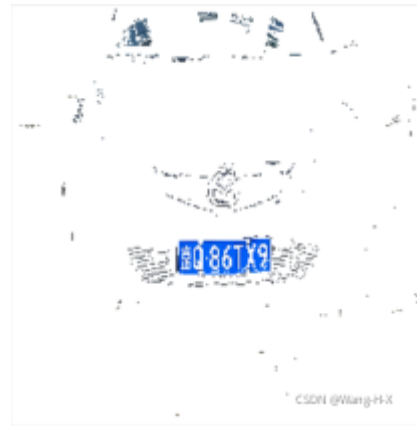
从图 2 中可以看出，经过一次图像处理操作，可以较好地车牌信息进行提取，但是由于环境的干扰，也会使得一些背景信息掺杂其中，所以需要根据小型车辆牌照“蓝底白字”的特点，提取图像中的蓝色部分，进一步过滤噪声。

针对图像处理输出的图像，首先根据 RGB 模型分理三个色彩通道的数据，存储在预设的矩阵中，然后遍历三个矩阵中的每一个像素点。通过设置判断条件，来辨别该像素点是否为蓝色。若为蓝色，则保持该位置数据不变；如非蓝色，则将像素点数据设置为 255。经过上述处理后，图像中蓝色区域会被保留，非蓝色区域则为白色。

此步骤的关键点在于判断条件的设置，由于输入照片色彩、光照条件不一，故难以找到可以满足所有图片的判断条件。在经过大量试验后，该系统设置判断条件为“像素点蓝色数值小于 65，并且红色、绿色数值大于 100”，该判断条件可以符合大多数场景。处理结果如图 4 所示。



(a) 第一次图像处理输出图



(b) 提取蓝色区域

图 4 提取蓝色区域效果图

```
% 对图像中蓝色区域进行提取
im6r = im6(:, :, 1);
im6g = im6(:, :, 2);
im6b = im6(:, :, 3);
for i = 1:m
    for j = 1:n
        if im6b(i, j) < 65 || im6r(i, j) > 100 || im6g(i, j) > 100
            pic(i, j, :) = 255;
        end
    end
end
im7 = pic;
subplot(3, 3, 5); imshow(im7);
```

4) 第二次图像处理

考虑到图像中，可能有除车牌以外的蓝色部分，或者满足上述判断条件的非车牌部分，会在后续的图像切割及字符识别过程中产生干扰，故设置第二次图像处理过程，该过程的与第一次图像处理使用的函数基本一致，结合实际情况，在具体参数选择上做出了改变。处理结果如图 5 所示



(a) 提取蓝色区域输出图



(b) 图像增强



(c) 图像二值化



(d) 形态学处理

图 5 二次图像处理效果图

```
% 提取新图象 S 通道信息
fig5 = rgb2hsv(im7);
im8 = fig5(:, :, 2);
subplot(3, 3, 6); imshow(im8);

% 增强新图像对比度, 并进行二值化
se2 = [0 1 0; 1 1 1; 0 1 0];
fig4 = imdilate(im8, se2);
fig5 = imdilate(fig4, se2);
im9 = imadjust(fig4);
im10 = im2bw(im9);
subplot(3, 3, 7); imshow(im10);
```

% 对提取图像进行膨胀处理, 进一步提取特征, 为保证下一步切割顺利, 再次进行闭运算。

```
fig2 = imdilate(im10, se2);
```

```
fig3 = imdilate(fig2, se2);
```

```
im11 = bwareaopen(fig3, 4000);
```

```
im12 = imclose(im11, se1);
```

```
subplot(3, 3, 8); imshow(im12);
```

虽然图 5 中的二次图像处理效果显著, 可以观察图 4-(a)中车牌周围存在大量的噪声信息, 经过二次图像处理后已经完全消失, 图 4-(d)中仅在车牌部分有白色的视觉效果。

综上所述, 经过图 2 所示的图像预处理流程后, 输入图像中的无用信息得到了充分的过滤, 关键的车牌信息保留了下来, 为后续的图像切割, 字符识别工作奠定了基础。

2.2 图像切割

系统输入图像经过预处理操作后, 车牌信息被保留, 无效信息最大程度被过滤掉。图像切割要完成的操作是根据预处理后的信息, 首先对车牌整体进行切割, 然后再对单个字符进行切割, 以便于下一步进行字符识别。

图像切割是本系统中最为重要的一环, 其流程图 (如图 6), 该部分输入为预处理完成后的图像, 输出为包含车牌信息的 7 个字符图像。该部分切割的准确度直接影响字符识别的准确度。如果切割环节出现了问题, 无论采取什么样的字符识别方法, 都将无济于事。接下来分别介绍车牌切割、字符粗切割、字符精切割三部分内容。

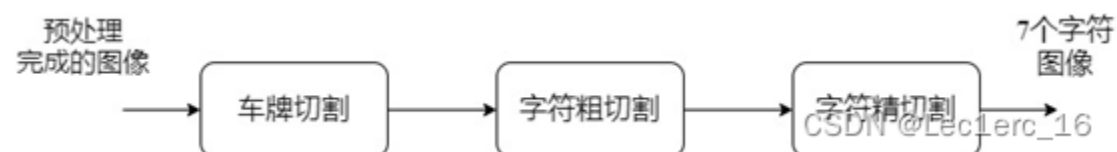


图 6 图像切割流程图

2.2.1 车牌切割

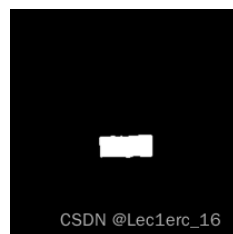
本部分实现的功能是在预处理图像的基础上, 对车牌进行定位切割。由图 5-(d)可以看出, 在经过预处理后, 图像中车牌位置显示为白色, 其余则显示为黑色, 但有些情况下非车牌位置可能也会有小面积的白色区域出现。车牌切割部分通过遍历图像的所有像素点, 设置判断条件, 将符合判断条件的像素点坐标值记录在数组中。记录完所有像素点的数据后, 在数组中搜索符合要求的数据, 从而确定切割点的位置, 运用

matlab 中的 imcrop 函数进行切割。

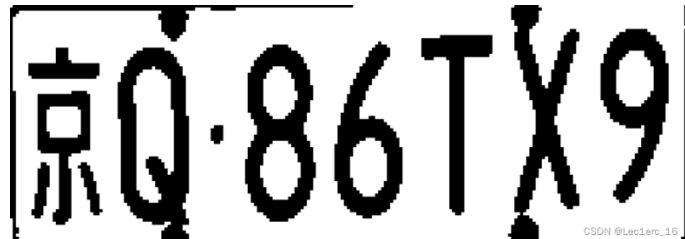
根据 imcrop 函数的使用方式分析，关键的坐标位置是车牌左上角与右下角。车牌左上角点位的特点是该像素点自身数据为 1，且其右下方有很多数据为 1 的像素点，故以此为基础，设置寻找车牌左上角坐标的判断条件。同理，车牌右下角的特点是该像素点自身数据为 1，且其左上角有很多数据为 1 的像素点，以此为基础，设置寻找车牌右下角坐标的判断条件。

在所有符合要求的坐标点中，左上角的坐标点横纵坐标之和必然是最小的，右下角的坐标横纵坐标之和必然是最大的。以此为依据，在数组中寻找满足要求的坐标点即可。考虑到汽车车牌有白色边框，可能会对下一步切割造成影响，所以将左上角坐标点稍向右下方向移动，将右下坐标点稍向左上方移动，从而得到车牌的切割点。

利用上述逻辑寻找图像中的车牌左上角点位与右下角点位，并进行切割，效果如图 7 所示。



(a) 预处理输出图像



(b) 车牌切割图

图 7 车牌切割效果图

```
% 寻找车牌位置
p1 = zeros(5000, 2);
h1 = 1;
for i = 1: m-51
    for j = 1: n-81
        if im12(i, j) == 1
            for k = 1:10
                if im12(i, j+k) == 1 && im12(i+k, j) == 1 && im12(i, j+70+k) == 1
                    && im12(i+40+k, j) == 1
                        p1(h1, 1) = i;
                        p1(h1, 2) = j;
                        h1 = h1+1;
                    end
                end
            end
        end
    end
end
```

```

        end
    end

    for i = 1:500
        if p1(i, 1) == 0 && p1(i, 2) == 0
            p1(i, 1) = m+n;
            % elseif abs(p1(i, 2) - n/2)*2/n > 0.95 || abs(p1(i, 1) - m/2)*2/m > 0.95
            %     p1(i, 2) = m+n;
        end
    end

    end

    s1 = sum(p1,2);
    min1 = min(s1);
    [row1, column1] = find(s1 == min1);
    X1 = p1(row1, 1);
    Y1 = p1(row1, 2);

    p2 = zeros(5000, 2);
    h2 = 1;
    for i = 51: m
        for j = 81: n
            if im12(i, j) == 1
                for k = 1:10
                    if im12(i, j-70+k) == 1 && im12(i-40+k, j) == 1 && im12(i, j-k) == 1
                        && im12(i-k, j) == 1
                            p2(h2, 1) = i;
                            p2(h2, 2) = j;
                            h2 = h2+1;
                        end
                    end
                end
            end
        end
    end

    end

    for i = 1:500

```

```

    if p2(i, 1) == 0 && p2(i, 2) == 0
        p2(i, 1) = -m-n;
        % elseif abs(p2(i, 2) - n/2)*2/n > 0.95 || abs(p2(i, 1) - m/2)*2/m > 0.95
        %     p2(i, 1) = -m-n;
    end
end

s2 = sum(p2,2);
max1 = max(s2);
[row2, column2] = find(s2 == max1);
X2 = p2(row2, 1);
Y2 = p2(row2, 2);

% 定义裁剪区域并进行裁剪
x1 = min(X1)+4;
y1 = min(Y1)+4;
x2 = max(X2)-4;
y2 = max(Y2)-4;
rect = [y1, x1, y2-y1, x2-x1];
im13 = imcrop(im4, rect);
figure(2)
imshow(im13)

```

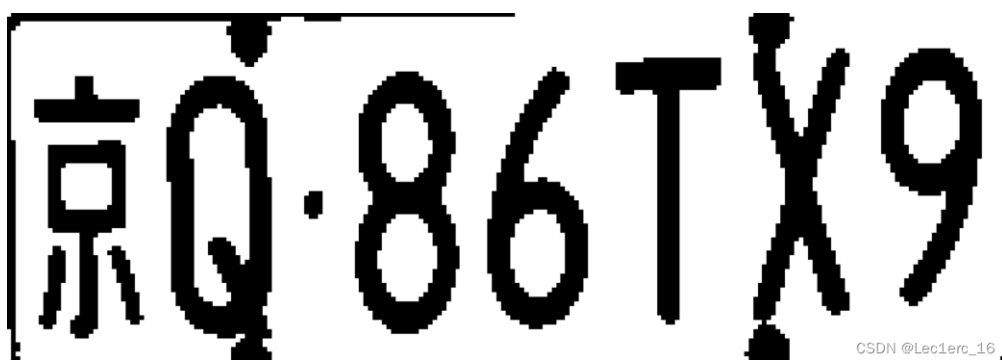
2.2.2 字符粗切割

本部分主要实现的功能是在车牌切割的基础上，依据中华人民共和国机动车号牌《GA 36-2014》中对于车牌字符位置的规定（如图 8），结合车牌切割输出图片的实际大小，利用 `imcrop` 函数对图片再次进行切割，将切割结果依次保存在三维矩阵中。

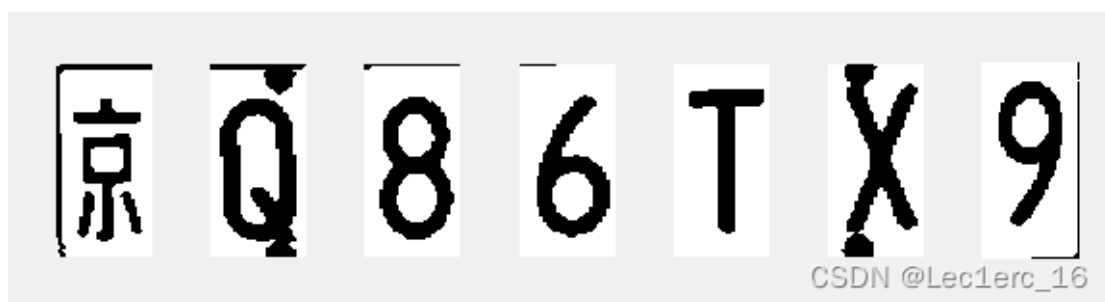


图 8 车牌标准样式图

根据上述标准进行字符粗切割的效果如图 9 所示。



(a) 车牌切割输出图像



(b) 字符粗切割

图 9 字符粗切割效果图

% 参照车牌国际标准，进行第一次分割，并将切割结果放入 im14 中

[m, n] = size(im13);

n1 = 0.1511*n;

n2 = 0.2807*n;

n3 = 0.3307*n;

n4 = 0.4602*n;

```

n5 = 0.5898*n;
n6 = 0.7193*n;
n7 = 0.8489*n;

rect1 = [0, 0, n1, m];
rect2 = [n1, 0, n2-n1, m];
rect3 = [n3, 0, n4-n3, m];
rect4 = [n4, 0, n5-n4, m];
rect5 = [n5, 0, n6-n5, m];
rect6 = [n6, 0, n7-n6, m];
rect7 = [n7, 0, n-n7, m];

figure(3)
pic1 = imresize(imcrop(im13, rect1), [100, 50]); im14(:, :, 1) = pic1(:, :);
subplot(1, 7, 1); imshow(im14(:, :, 1));
pic2 = imresize(imcrop(im13, rect2), [100, 50]); im14(:, :, 2) = pic2(:, :);
subplot(1, 7, 2); imshow(im14(:, :, 2));
pic3 = imresize(imcrop(im13, rect3), [100, 50]); im14(:, :, 3) = pic3(:, :);
subplot(1, 7, 3); imshow(im14(:, :, 3));
pic4 = imresize(imcrop(im13, rect4), [100, 50]); im14(:, :, 4) = pic4(:, :);
subplot(1, 7, 4); imshow(im14(:, :, 4));
pic5 = imresize(imcrop(im13, rect5), [100, 50]); im14(:, :, 5) = pic5(:, :);
subplot(1, 7, 5); imshow(im14(:, :, 5));
pic6 = imresize(imcrop(im13, rect6), [100, 50]); im14(:, :, 6) = pic6(:, :);
subplot(1, 7, 6); imshow(im14(:, :, 6));
pic7 = imresize(imcrop(im13, rect7), [100, 50]); im14(:, :, 7) = pic7(:, :);
subplot(1, 7, 7); imshow(im14(:, :, 7));

```

2.2.3 字符精切割

本部分主要实现的功能是在字符粗切割的基础上，进一步进行切割，得到无边缘的字符图像（字符占满整个图像空间）。字符精切割这一步骤对基于模板匹配的字符识别意义重大。

由于车牌边框的存在，粗切割后的 7 张图像边缘或多或少都有黑色边缘出现，分析图像可知，第 1 张图像左边缘、上边缘、下边缘均可能出现黑边，第 2 到第 6 张图

像只有上下可能出现黑边，第 7 张图像右边缘、上边缘、下边缘可能出现黑边。为了提高切割的精确度，同时又避免程序复杂引起的时效性下降，决定采取如下处理方法。

第 2 到第 6 张图片均为字母或数字，是连续的结构，且仅在上下边缘有黑边。所以采取“先上下，后左右，从中间，到两边”的方法进行切割。“先上下，后左右”，意思是先寻找切割的上下边界，并予以切割，再寻找切割的左右边界，再次进行切割。“从中间，到两边”，意思是每次搜索都从图像的中间向两边遍历，因为字母与数字都是连续结构，故可以使用这一方法，在一定程度上也避免了边缘黑边的影响。由于这些粗切割图片上下边缘有黑边，左右边缘没有黑边，所以从图像中间一行开始，向上方搜索每一行的数据，图像中白色像素点数据为 1，黑色像素点数据为 0，每次遇到黑色像素点，计数器就加 1。如此看来，当到达某一行时计数器变为 0（或小于某一预设值，增强适用性），则可以判断该行没有黑色像素点存在，则该行即为切割的上边界。同理，可以找到切割的下边界。利用 `imcrop` 函数即可完成对图像上下空白区域及黑色边缘的切割。此时图像上下已经没有黑色边缘影响，与寻找上下边界相同，可以参照上述原理寻找图像左右边界，再利用 `imcrop` 函数即可完成图像精切割。

第 7 张图像同样为数字或字母，但与前面不同的是，它有更复杂的黑色边缘干扰，所以应先切除黑色边缘，再采取与前面相同的精切割方法提取字符。切除黑色边缘采用从两边到中间的方法，从最上面一行开始，向下搜索每一行的数据，每次遇到黑色像素点，计数器加 1，当计数器数值为 0（或小于某一预设值，增强适用性），则可认为已经排除完黑色边缘，则设置此行为切割上限，同理可以寻找下限，进行一次切割。再类比上述方法寻找左右的切割线，完成去除黑色边缘的操作。去除黑色边缘后，按照第 2 到第 6 张图像的切割方法即可完成第 7 张图像的精切割。

第 1 张图像有复杂的黑色边缘，且字符为汉字。首先进行黑色边缘的切除，采用的切除方法与第 7 张图像的切除方法一致，完成切除后，准备对字符进行精切割，由于汉字并不是连续的结构，不能采取“从中间，到两边”的切割方法，所以我们使用“从两边，到中间”的常规搜索方法，具体的实施办法与“从中间，到两边”类似，只是更改了搜索的方向，判断条件也做出相应的调整即可，在此不再赘述。最终将切割完成后的 7 张图像保存在 3 维矩阵中即可。

对字符进行精切割后的效果如图 10 所示。



(a) 字符精切割输入图



(b) 字符精切割输出图

图 10 字符精切割效果图

for k = 1: 7

% 首先对首尾字符进行处理

if k == 1 || k == 7

piccut = im14(:, :, k);

[m, n] = size(piccut);

m1 = 0; m2 = 0;

%搜索裁剪的上边界

for i = 1: 1: m/2

count1 = 0;

for j = 1: n

if piccut(i, j) == 0

count1 = count1+1;

end

end

if count1 <= 6

m1 = i+4;

break

end

end

%搜索裁剪的下边界

for i = m: -1: m/2

count2 = 0;

for j = 1: n

if piccut(i, j) == 0

```

        count2 = count2+1;
    end
end
if count2 <= 6
    m2 = i-4;
    break
end
end
if m2 == 0;
    m2 = m;
end

```

% 首先根据行数，做高度方向的裁剪

```

rect10 = [0, m1, n, m2-m1];
im15 = imcrop(piccut, rect10);
[m, n] = size(im15);
n1 = 0; n2 = 0;

```

% 搜索裁剪的左边界

```

for j = 1: 1: n/2
    count3 = 0;
    for i = 1: m
        if piccut == 0
            count3 = count3+1;
        end
    end
    if count3 <= 6
        n1 = j+4;
        break
    end
end
end

```

% 搜索裁剪的右边界

```

for j = n: -1: n/2
    count4 = 0;

```

```

        for i = 1: m
            if piccut(i, j) == 0
                count4 = count4+1;
            end
        end
        if count4 <= 6
            n2 = j-4;
            break
        end
    end
    if n2 == 0;
        n2 = n;
    end

    % 对图像进行二次处理
    rect11 = [n1, 0, n2-n1, m];
    im16 = imresize(imcrop(im15, rect11), [100,50]);
    im14(:, :, k) = im16;
end

% 搜索图像切割边界
picpro = im14(:, :, k);
[m, n] = size(picpro);
m1 = 0; m2 = 0;

if k == 1

    % 搜索切割的上边界
    for i = 1: 1: m/2
        count1 = 0;
        for j = 1: n
            if picpro(i, j) == 0
                count1 = count1+1;
            end
        end
    end

```

```

        if count1 >= 5
            m1 = i+1;
            break
        end
    end
end

% 搜索切割的下边界
for i = m: -1: m/2
    count2 = 0;
    for j = 1: n
        if picpro(i, j) == 0
            count2 = count2+1;
        end
    end
    if count2 >= 5
        m2 = i-1;
        break
    end
end
if m2 == 0
    m2 = m;
end

```

```

% 首先根据行数，做高度方向的裁剪
rect12 = [0, m1, n, m2-m1];
im17 = imcrop(picpro, rect12);
[m, n] = size(im17);
n1 = 0; n2 = 0;

```

```

% 寻找切割的左边界
for j = 1: 1: n/2
    count3 = 0;
    for i = 1: m
        if im17(i, j) == 0
            count3 = count3+1;
        end
    end
end

```

```

        end
    end
    if count3 >= 6
        n1 = j+1;
        break
    end
end

% 寻找切割的右边界
for j = n: -1: n/2
    count4 = 0;
    for i = 1:m
        if im17(i, j) == 0
            count4 = count4+1;
        end
    end
    if count4 >= 6
        n2 = j-1;
        break
    end
end
if n2 == 0;
    n2 = n;
end

```

```

% 完成切割工作
rect13 = [n1, 0, n2-n1, m];
im18 = imresize(imcrop(im17, rect13), [40, 20]);
fipic(:, :, k) = im18;

```

```

else
    % 搜索切割的上边界
    for i = m/2: -1: 1
        count1 = 0;
        for j = 1: n

```

```

        if picpro(i, j) == 0
            count1 = count1+1;
        end
    end
    if count1 <= 5
        m1 = i+1;
        break
    end
end

```

% 搜索切割的下边界

```

for i = m/2: 1: m
    count2 = 0;
    for j = 1: n
        if picpro(i, j) == 0
            count2 = count2+1;
        end
    end
    if count2 <= 5
        m2 = i-1;
        break
    end
end
if m2 == 0
    m2 = m;
end

```

% 首先根据行数，做高度方向的裁剪

```

rect12 = [0, m1, n, m2-m1];
im17 = imcrop(picpro, rect12);
[m, n] = size(im17);
n1 = 0; n2 = 0;

```

% 寻找切割的左边界

```

for j = n/2: -1: 1

```

```

count3 = 0;
for i = 1: m
    if im17(i, j) == 0
        count3 = count3+1;
    end
end
if count3 <= 6
    n1 = j+1;
    break
end
end

```

% 寻找切割的右边界

```

for j = n/2: 1: n
    count4 = 0;
    for i = 1:m
        if im17(i, j) == 0
            count4 = count4+1;
        end
    end
    if count4 <= 6
        n2 = j-1;
        break
    end
end
if n2 == 0
    n2 = n;
end

```

% 完成切割工作

```

rect13 = [n1, 0, n2-n1, m];
im18 = imresize(imcrop(im17, rect13), [40, 20]);
fipic(:, :, k) = im18;
end

```

```

%创建路径保存并展示
name = 'VLPR';
datasavename = [name datestr(now, 5) datestr(now, 7)];
mkdir(datasavename);
save(['.\' datasavename '\',num2str(k),'.jpg'],'im18')
figure(4)
subplot(1, 7, k); imshow(im18);
end

```

2.3 字符识别

本系统采用模板匹配的方法实现字符识别。模板匹配的首要任务是建立字符模板库，在网络上搜集相关字符图片，利用 modelget.m 程序将图片转化为系统所需要的“白底黑字”样式，最终完成了数字 0-9、字母 A-Z、以及汉字，共计 67 个字符的模板建立。对字符模板进行编号，其中数字 0-9 对应序号为 1-10，字母 A-Z（没有 O、I）对应序号为 11-34，汉字对应序号为 35-65，建立字符与序号对应的表格 codebook.xlsx、codebooknum.xlsx。根据中华人民共和国机动车号牌标准《GA 36-2014》5.8 序号编码规则和使用规则规定，字母 O 与 I 不能出现在车牌中，所以不需要设置特定的程序对 0 和 O、1 和 I 加以区分。

进行字符识别时，首先根据图像特征设定搜索模板的范围。第 1 张图像为汉字字符，故模板序号对应的搜索范围为 35-65，第 2 张图像为英文字母，故模板序号对应的搜索范围为 11-34，第 3 张到第 7 张图像为字母或数字，故模板序号对应的搜索范围为 1-34。将要识别的字符图像分别与搜索范围内的模板的像素点一一进行比对，记录一致的像素点的个数，并将该数据存储在数组的对应位置中。当要识别的字符图像与搜索范围内的模板全部完成比对，且均已将比对数据记录在数组内后，数组内最大值对应的模板就是该图像最有可能的真实字符。如此循环七次即可得到图像中的车牌数据，最终将识别出的车牌数据输出即可（如图 11 所示）。



图中车牌为：京Q86TX9

CSDN @Lec1erc_16

图 11 字符识别效果图

2.4 总结

综上所述，系统输入图片经过图像预处理、图像分割、字符识别三大步骤，即可识别出图像中的车牌信息，并予以输出。实现了车牌识别系统的功能，满足“项目背景及目标”中提出的项目目标。

3 系统的局限性

3.1 汉字字符的识别

为了防止车牌的伪造，所以我国车牌字体并未公开，所以字符模板均为本人在互联网上搜索图片，并自行制作完成，与标准字体仍有一定差距。在图像处理过程中，由于拍摄图片的质量层次不齐，经过预处理、图像切割等步骤后，汉字字符可能会变得较为模糊。字符识别时，汉字字符不清晰，汉字模板也并不标准，可能导致车牌中的汉字识别失败。

3.2 蓝色车牌的识别

在图像预处理过程中比较重要的一步是，提取图像中的蓝色区域，为此设置了“像素点蓝色数值小于 65，并且红色、绿色数值大于 100”的判断条件，但如果环境中有多与车牌颜色相同的元素（如蓝色车辆），则会使该步骤失效，从而导致无法识别出图像中的车牌信息。

3.3 UI 界面的设计

由于时间紧迫，考试周临近，并没有为该系统设计合适的 UI 界面，使得该系统操作起来较为生硬、麻烦、不直观。